

Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky

Adaptívne odporúčanie študijných akcií pre personalizovanú výučbu

Bakalárska práca

PRÍLOHA B

Systémová príručka

Študijný program: Inteligentné systémy
Študijný odbor: Informatika
Školiace pracovisko: Katedra kybernetiky a umelej inteligencie (KKUI)
Školiteľ: Ing. Ján Magyar, PhD.

Košice 2025

Dávid Firda

Obsah

1	Účel a prehľad systému	2
2	Súpis obsahu dodávky	3
3	Spracovanie dát a príprava datasetu	4
3.1	Kategorizácia otázok a generovanie nesprávnych odpovedí	4
3.2	Príprava vstupných údajov a validácia	5
4	Popis a implementácia systému	6
4.1	Náhodný výber otázok (Random)	6
4.2	Q-learning – posilňovacie učenie	6
4.3	POMDP – Bayesovský prístup s Thompson Sampling	7
4.4	Simulácie a experimentálne overenie	8
4.4.1	Parametre simulácie	8
4.4.2	Priebeh a scenáre testovania	8
4.4.3	Vizualizácia výsledkov a hodnotenie	9
5	Popis aplikácie	12
5.1	Požiadavky na systém	12
5.2	Základná štruktúra projektu	12
5.3	Backend	13
5.3.1	Algoritmy	14
5.4	Frontend	15
5.5	Administrácia	15
6	Spustenie aplikácie	16
	Zoznam použitej literatúry	18

1 Účel a prehľad systému

Vytvorený systém je určený pre adaptívne testovanie znalostí programovania u užívateľov so zameraním na programovací jazyk Python. Primárnym cieľom je, aby dokázal vybrať najvhodnejšiu otázku podľa predchádzajúceho výkonu používateľa, čo znamená, že sa musí prispôbiť jeho slabým a silným stránkam. Systém je teda navrhnutý tak, aby v reálnom čase vyhodnocoval odpovede študenta a podľa toho upravoval výber ďalších úloh. Vývoj prebiehal v dvoch fázach:

1. Prvá fáza

Vytvorenie simulácie adaptívneho testovania, čo umožnilo prezentáciu a kontrolu funkcie výberu v pilotnom prostredí. Vytvorenie simulovaného študenta, rôzne metódy výberu úloh, ako sú náhodný výber, Q-learning a POMDP, a súbor experimentov na ich testovanie.

2. Druhá fáza

Návrh webovej aplikácie využívajúcej výsledky z prvej fázy projektu. Po registrácii používateľ absolvuje predtest, po ktorom nasleduje hlavný test obsahujúci 30 otázok. Počas testu systém pamätá odpovede používateľa, ktoré následne vyhodnocuje a po teste poskytuje spätnú väzbu. Nakoniec aplikácia zahŕňa aj spätnú väzbu a dotazník na posúdenie používateľskej skúsenosti.

2 Súpis obsahu dodávky

Na priloženom elektronickom médiu (Príloha C) sa nachádza kompletný obsah projektu, ktorý bol vypracovaný v rámci bakalárskej práce. Obsah dodávky je rozdelený do viacerých logických celkov zodpovedajúcich jednotlivým fázam vývoja systému:

- `dataset_processing` – skripty a notebooky na čistenie a kategorizáciu otázok, validáciu výstupov a generovanie nesprávnych odpovedí.
- `simulations` – Python skripty na testovanie algoritmov v simulovanom prostredí vrátane grafického porovnania ich výkonu.
- `student_testing_app` – kompletný systém adaptívneho testovania:
 - **backend** – serverová Flask aplikácia s REST API, výberom otázok podľa algoritmov (Random, Q-learning, POMDP), databázovým modelom a spracovaním odpovedí,
 - **frontend** – responzívne HTML/CSS/JavaScript rozhranie pre študentov a administrátorov,
 - `pomdp_data`, `q_learning_data` – priebežné súbory s údajmi algoritmov počas testovania,
 - `final_dataset.csv` – finálny dataset použitý počas reálneho testovania.
 - konfiguračné súbory – `.env`, `docker-compose.yml`, `Dockerfile`, `init.sql`, `load_questions.py` potrebné na nasadenie aplikácie pomocou Dockeru.
- Používateľská príručka
- Systémová príručka

3 Spracovanie dát a príprava datasetu

Táto časť je dôležitá pre poskytnutie testovacej a tréningovej dátovej sady. Ako prvé potrebné predspracovanie surových dát získaných z datasetu (1): `Python Programming Questions Dataset.csv`, ktorá prebiehala v jupyter notebooku `rawdata_analysis.ipynb`. Týmto sme chceli odhaliť neúplné alebo nekonzistentné záznamy, odstrániť príliš dlhé úlohy či také, ktoré sú pre náš účel nerelevantné.

3.1 Kategorizácia otázok a generovanie nesprávnych odpovedí

Následne boli otázky zaradené do tematických kategórií a podkategórií. Tento proces bol realizovaný v skripte `data_categorization.py`, ktorý využíva kombináciu čistenia textu a vyhľadávania pomocou kľúčových slov. Funkcia `categorize_questions` najskôr aplikuje predspracovanie vstupného textu – odstráni neželané znaky (`clean_text`) a zjednoduší slová do jednotného tvaru (`to_singular`), čo zvyšuje spoľahlivosť pri porovnávaní s definovanými zoznamami kľúčových slov. Na základe nich funkcia `determine_category` prehľadáva všetky možné slovné spojenia v inštrukcii a porovnáva ich s kľúčovými slovami, ktoré sú priradené ku každej z kategórií. Ak dôjde k zhode, otázke sa priradí zodpovedajúca kategória a podkategória. Otázky, ktoré sa nepodarilo takto zatriediť, sú ďalej spracované funkciou `categorize_uncategorized_by_output`, ktorá skúma a prehľadáva ich očakávaný výstup. Zmyslom tohto kroku je maximalizovať pokrytie kategóriami aj pre otázky, ktorých textová formulácia neobsahovala zjavné kľúčové slová. Výsledkom celého procesu je doplnenie nových stĺpcov `Category` a `Subcategory` do pôvodného datasetu.

Dôležitou súčasťou úpravy datasetu bola aj generácia nesprávnych odpovedí. V skripte `generation_Incorrect_output.py` bol ku každej otázke priradený stĺpec `IncorrectOutput`, ktorý obsahoval syntakticky podobnú, no nesprávnu odpoveď. Funkcia `generate_incorrect_output()` vyberá podľa podkategórie úlohy typické

chyby, ktoré môžu študenti spraviť – preklepy, zlá syntax alebo nesprávne príkazy.

3.2 Príprava vstupných údajov a validácia

Po kategorizácii nasledovalo otestovanie správnosti výstupov. V skripte `test_spustitelnosti.py` bol vytvorený mechanizmus na spúšťanie kódu (funkcia `capture_output`), ktorý filtroval otázky s nefunkčným výstupom. Použitím `exec()` sa zisťovalo, či výstup kódu v stĺpci `Output` je spustiteľný, pričom všetky záznamy spôsobujúce chyby boli vyradené. Tento krok odstránil otázky s neplatným kódom (napr. syntax error alebo zablokovanie pri vstupe) a roztriedil otázky do skupín:

- `runnable_with_output` – otázky s funkčným kódom a výstupom,
- `runnable_no_output` – otázky, ktoré sa síce spustili, ale nič nevypísali,
- `not_runnable` – otázky spôsobujúce chyby pri spustení.

Po vykonaní čistenia bol vytvorený finálny dataset `final_dataset.csv`, ktorý už obsahoval len validné a tematicky zatriedené otázky.

4 Popis a implementácia systému

Kapitola popisuje implementáciu jednotlivých prístupov výberu otázok, ktoré boli testované počas simulácie aj v reálnom prostredí. Všetky algoritmy boli implementované v jazyku Python a integrované do spoločného systému výučby.

Študent v simulácii je reprezentovaný triedou `StudentMemory` (`EFCStudent_model.py`). Trieda `Record` zaznamenáva odpovede pre každú kategóriu a z nich odvodzuje pravdepodobnosť správnej odpovede pri ďalšom pokuse.

4.1 Náhodný výber otázok (Random)

Kód, ktorý vyberá otázky náhodne, funguje v súbore `RandomQuestionSelector.py`. Musí náhodne vyberať z jednej z možných kategórií otázok nezávisle od predošlého výkonu študenta. Hlavná funkcia `select_category()` využíva `random.choice()`, ktorá náhodne vyberá kategóriu zo zoznamu.

4.2 Q-learning – posilňovacie učenie

Algoritmus Q-learning bol implementovaný v súbore `Qlearning.py`. Každá kategória otázok je považovaná za stav a výber ďalšej otázky predstavuje akciu. Pre každý pár stav-akcia sa udržiava Q-hodnota, ktorá reprezentuje očakávanú odmenu.

- `Select_category()` využíva *epsilon-greedy* stratégiu: náhodná kategória sa vyberie s pravdepodobnosťou ϵ , inak kategória s najvyššou Q-hodnotou. Systém navyše sleduje počet nesprávnych odpovedí za sebou – ak študent trikrát zlyhá v jednej kategórii, ďalšia otázka bude zo silnejšej oblasti.
- Funkcia `update_q_value()` aktualizuje Q-hodnoty podľa klasického vzorca Q-learningu, pričom odmena závisí od správnosti odpovede a toho, či ide o slabú kategóriu.
- Systém používa inverzný systém odmeňovania: za správne odpovede priraduje záporné hodnoty (napr. -10 , -15), zatiaľ čo za nesprávne odpovede kladné

(napr. +5, +10). Tento mechanizmus spôsobí, že systém preferuje kategórie, kde sa študentovi nedarí – a teda prirodzene smeruje otázky do slabších oblastí.

- Úspešnosť v kategóriách sa priebežne vyhodnocuje pomocou `is_weak_category()`, kde je hranica slabosti nastavená na 60 % správnych odpovedí. Výber otázok a pridelovanie odmiern sú tomu dynamicky prispôbené.

4.3 POMDP – Bayesovský prístup s Thompson Sampling

Prístup POMDP bol implementovaný v troch verziách, pričom finálny model (`POMDP_v3.py`) využíva bayesovskú stratégiu založenú na *Thompson Sampling*. Porovnanie verzií ukázalo, že práve tento prístup bol najefektívnejší pri cieleť adresovaní slabých stránok študenta.

Model je tvorený viacerými komponentmi:

- `TutorState` reprezentuje stav študenta v rámci jednej kategórie. Stavby sú diskrétny (napr. „nízka“, „stredná“, „vysoká“ znalosť) a slúžia na modelovanie pokroku študenta v čase.
- `TutorTransitionModel` definuje, ako sa stav študenta zmení po vykonaní akcie – teda po položení otázky. Pravdepodobnosť prechodu medzi stavmi zohľadňuje predchádzajúci výkon a odpoveď na aktuálnu otázku.
- `TutorObservationModel` modeluje pravdepodobnosť správnej odpovede v danom stave, vzhľadom na jeho (skrytý) stav vedomostí. Tento model tvorí základ pre aktualizáciu presvedčenia (*belief update*) po každej odpovedi.
- `TutorBelief` uchováva pravdepodobnostnú distribúciu (*belief*) pre každú kategóriu a aktualizuje ju po každej odpovedi.
- `TutorPolicyModel` rozhoduje, ktorú kategóriu vybrať v ďalšom kroku. Používa výber pomocou `softmax` nad skóre kategórií, ktoré kombinuje entropiu (ako mieru neistoty) a predpokladanú úspešnosť. Vysoká neistota znamená, že

o znalostiach študenta v danej kategórii ešte nie je dostatok informácií – čo zvyšuje prioritu jej výberu.

Akcia, teda výber kategórie, sa vyberá podľa Thompson Sampling – vzorkovaním z beta distribúcie pre každú kategóriu sa náhodne vzorkuje pravdepodobnosť správnej odpovede a následne sa vyberá kategória s najnižším skóre.

4.4 Simulácie a experimentálne overenie

Simulácie predstavovali dôležitý krok pri overení funkčnosti navrhnutých algoritmov pred ich nasadením do reálneho prostredia. Ich cieľom bolo testovať schopnosť jednotlivých prístupov adaptívne reagovať na výkonnosť študenta a cielene precvičovať jeho slabé stránky.

4.4.1 Parametre simulácie

Simulovaný študent bol reprezentovaný modelom, ktorý udržiaval informácie o úspešnosti v jednotlivých kategóriách. Pred začiatkom simulácie boli pre každého študenta náhodne zvolené slabé kategórie. Pravdepodobnosť správnej odpovede bola definovaná oddelene pre silné a slabé kategórie a mohla byť nastavená pomocou parametrov simulácie.

Každý algoritmus (Random, Q-learning, POMDP) bol testovaný na rovnakej sade otázok. Počet otázok, počet študentov, ako aj výber kategórií bol identický, aby sa zabezpečila porovnateľnosť výsledkov.

4.4.2 Priebeh a scenáre testovania

Simulácia prebiehala iteratívne – pre každého študenta sa v každej iterácii vybrala jedna kategória na základe stratégie daného algoritmu. Následne bola vygenerovaná odpoveď podľa simulovanej pravdepodobnosti a algoritmus aktualizoval svoje interné hodnoty (napr. Q-hodnoty alebo belief stavy). Každý skript simulácie (napr. `Simulation_Qlearning.py`, `Simulation_POMDP.py`) inicializoval zoznam kategórií

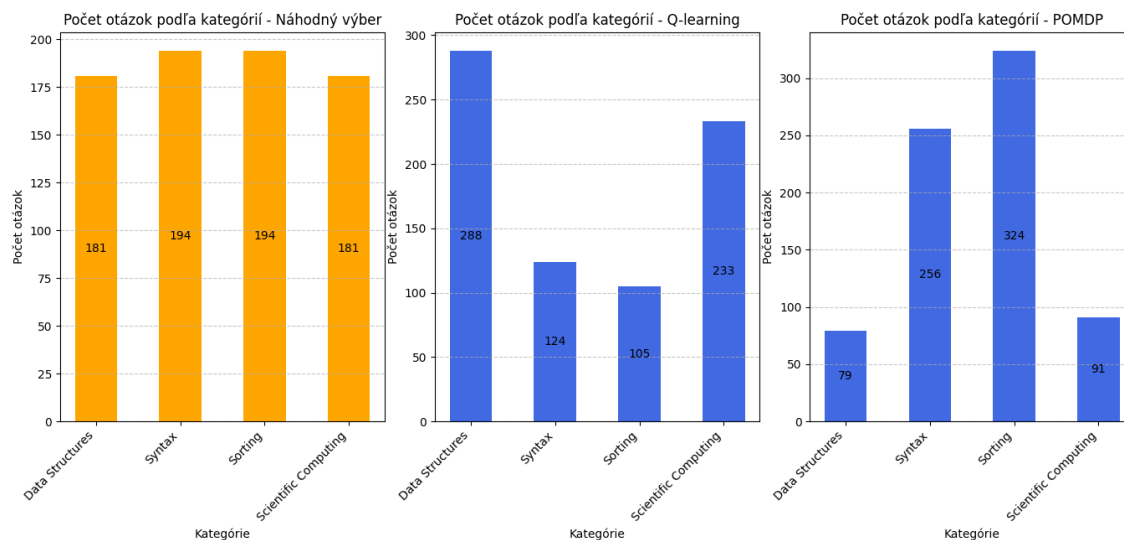
a pre každého študenta vytvoril samostatnú inštanciu simulácie. V každej simulácii sa zaznamenávali:

- úspešnosť v slabých a silných kategóriách,
- vývoj skóre počas iterácií,
- počet otázok položených v jednotlivých kategóriách,
- výkon algoritmu pri cielej intervencii.

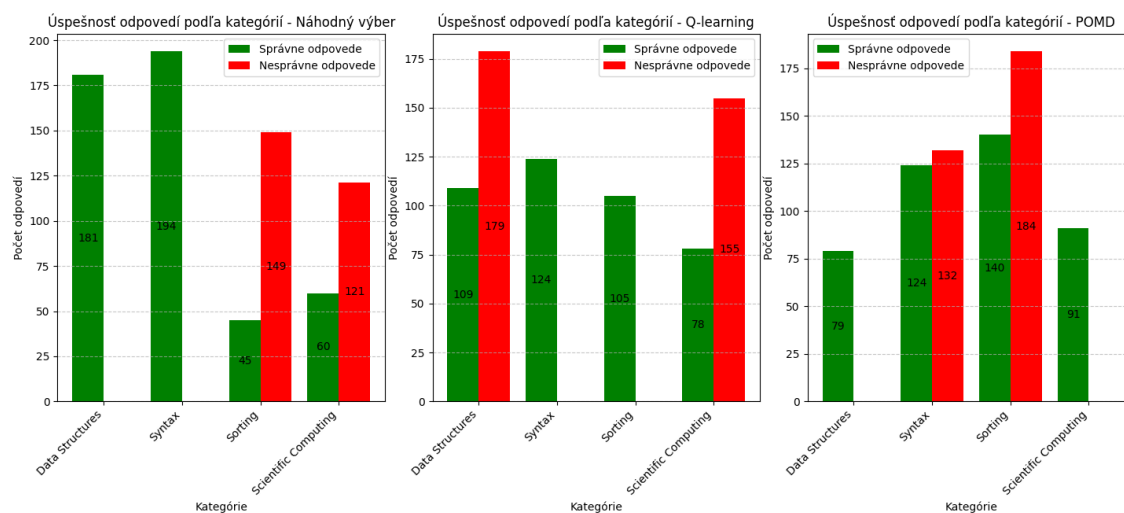
4.4.3 Vizualizácia výsledkov a hodnotenie

Výstupom každej simulácie boli súbory obsahujúce podrobné logy výkonnosti (napr. CSV alebo JSON), ktoré boli spracované do prehľadných grafov pomocou funkcie `visualize_results()`. Tieto vizualizácie zobrazovali:

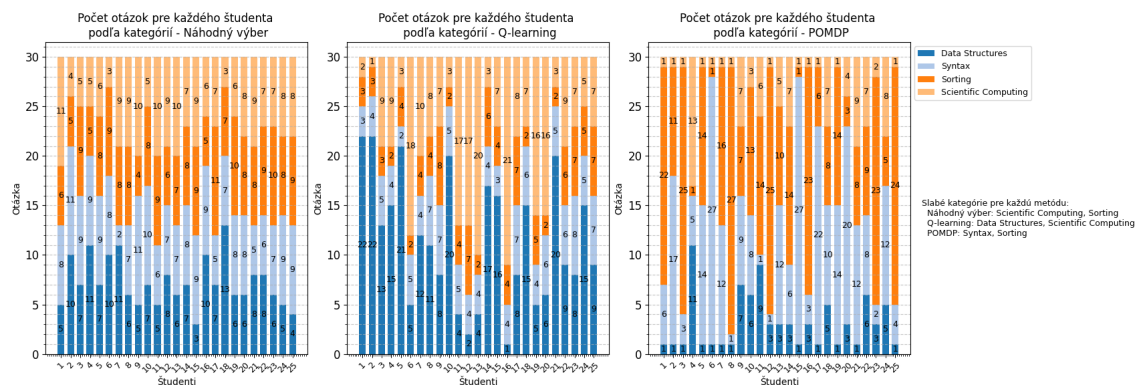
- ako často sa algoritmus zameral na kategórie (obrázok 4–1 a 4–2),
- počet položených otázok na základe kategórií pre každého respondenta zvlášť (Obr. 4–3),
- porovnanie medzi jednotlivými prístupmi Obr. 4–4).



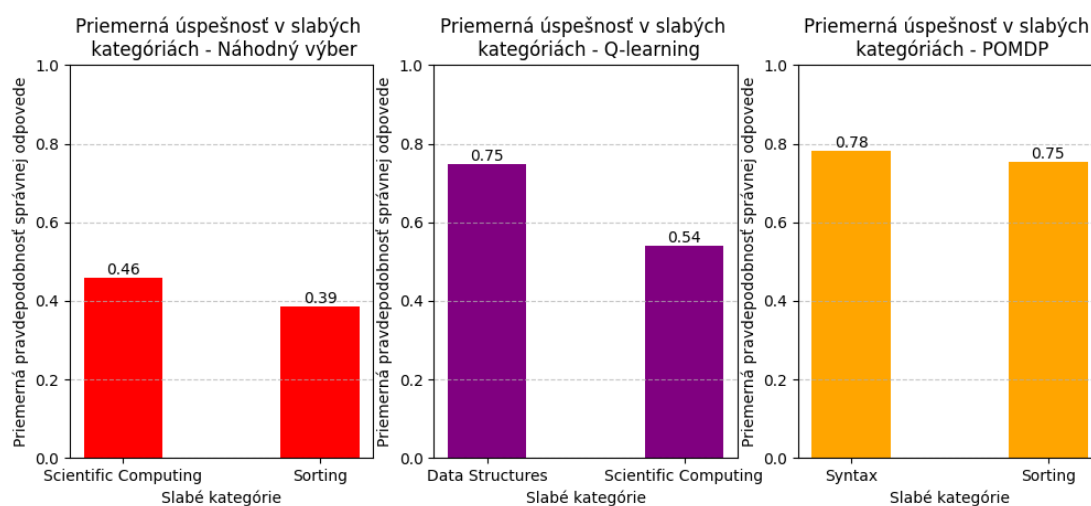
Obr. 4–1 Rozdelenie počtu otázok podľa kategórií pre každého študenta pri rôznych prístupoch



Obr. 4–2 Priemerná úspešnosť v kategóriách pre jednotlivé algoritmy



Obr. 4–3 Počet otázok položených v jednotlivých kategóriách pre každý prístup



Obr. 4–4 Správnosť odpovedí podľa slabých kategórií pre jednotlivé algoritmy

5 Popis aplikácie

Táto kapitola popisuje štruktúru a fungovanie aplikácie **EduGuide**. Aplikácia je rozdelená na frontendovú a backendovú časť a komunikuje prostredníctvom REST API. Projekt podporuje spustenie cez Docker (2) a je konfigurovaný pre PostgreSQL (3) databázu.

5.1 Požiadavky na systém

Pre úspešné nasadenie systému je potrebné mať v systéme dostupné nasledovné komponenty:

- **operačný systém:** Windows 10/11, Linux alebo macOS s podporou virtualizácie,
- **docker:** verzia 20.10 alebo vyššia,
- **docker compose:** verzia 1.29+ (alebo v rámci Docker CLI),
- **sieťové pripojenie:** pre prístup k balíčkam a prípadné sťahovanie závislostí,
- **systémové prostriedky:** minimálne 4 GB RAM a 500 MB voľného miesta na disku.

Nie je nutné mať predinštalované Python, PostgreSQL ani žiadne ďalšie externé služby – všetko je súčasťou kontajnerov.

5.2 Základná štruktúra projektu

Na koreňovej úrovni projektu sa nachádzajú konfiguračné súbory:

- **.env** – obsahuje environmentálne premenné,
- **docker-compose.yml**, **Dockerfile** – definujú kontajnerové prostredie pre backend, frontend a databázu,

- `init.sql` – skript na inicializáciu databázových tabuliek,
- `final_dataset.csv` – finálny dataset otázok použitý počas testovania,
- `load_questions.py` – skript pre nahratie datasetu do databázy.

5.3 Backend

Backend je umiestnený v priečinku `backend/` a je vytvorený pomocou frameworku Flask (4). Jeho štruktúra je nasledovná:

- `app.py` – hlavný spúšťač aplikácie,
- `api_routes.py`, `admin_routes.py` – obsahujú definíciu API koncových bodov (Tabuľka 5–1) pre používateľské aj administrátorské rozhranie,
- `models.py` – obsahuje dátové modely a databázové tabuľky,
- `capture_output.py`, `extract_starter_code.py` – pomocné moduly na spracovanie odpovedí a študentského kódu,
- `config.py`, `__init__.py` – inicializácia a nastavenie aplikácie.

Cesta (Route)	Metóda	Popis
/api/	GET	Kontrola funkčnosti API a úvodná stránka.
/api/register	POST	Registrácia nového študenta do systému.
/api/login	POST	Prihlásenie existujúceho študenta.
/api/test/start	POST	Spustenie predtestu s vybranými otázkami.
/api/main_test/start	POST	Spustenie hlavného testu na základe priradeného algoritmu.
/api/test/answer	POST	Vyhodnotenie odpovede študenta a aktualizácia modelu.
/api/test/analysis	POST	Zobrazenie analýzy výsledkov po ukončení testu (úspešnosť, percentil).
/api/feedback	POST	Odoslanie spätnej väzby od študenta po testovaní.
/api/feedback/check	POST	Overenie, či už bola spätná väzba vyplnená.
/admin/students	GET	Export všetkých študentov a výsledkov do Excel súboru.
/admin/students/summary	GET	Prehľadné štatistiky pre všetkých študentov.
/admin/algorithms/stats	GET	Zhrnutie výkonnosti algoritmov a ich úspešnosti.
/admin/feedback	GET	Export spätnej väzby do Excel súboru.
/admin/feedback/summary	GET	Zhrnutie všetkých odpovedí z dotazníka.

Tabuľka 5 – 1 Prehľad API koncových bodov aplikácie

5.3.1 Algoritmy

Algoritmy výberu otázok sú umiestnené v priečinku `algorithms/` a zahŕňajú:

- `random_selector.py` – náhodný výber otázok z celého datasetu,
- `q_learning.py`, `q_selector.py` – implementácia posilňovacieho učenia a stra-

tégie výberu kategórií,

- `pomdp.py`, `pomdp_selector.py` – model na báze POMDP a výber kategórie pomocou Thompson Sampling.

5.4 Frontend

Frontend sa nachádza v priečinku `frontend/` a je postavený na technológiách HTML, CSS a JavaScript. Používa knižnicu `Chart.js` na tvorbu vizualizácií. Medzi hlavné komponenty patria:

- `index.html`, `register.html`, `login.html` – úvodné a prihlasovacie obrazovky,
- `predtest.html`, `hlavnytest.html` – realizácia predtestu a hlavného testu,
- `analiza.html` – analýza výkonu po ukončení testu,
- `feedback.html`, `feedback_visualization.html` – dotazník spokojnosti a zobrazenie spätnej väzby,
- `algorithm_comparison.html` – vizualizácia výkonnosti algoritmov.

Každá HTML stránka má priradený JavaScript súbor, ktorý zabezpečuje dynamické správanie a API komunikáciu.

5.5 Administrácia

Administrátorské API poskytuje prehľad o výkonnosti študentov, spätných väzbách a úspešnosti algoritmov. Získané dáta sú vizualizované vo `feedback_visualization.html` a `algorithm_comparison.html`, kde je možné analyzovať výkonnosť systémov a výsledky simulácií a reálneho testovania.

6 Spustenie aplikácie

Aplikácia vyžaduje inštaláciu nástroja Docker (2), aby sa mohla spustiť. Následne postupujte podľa krokov:

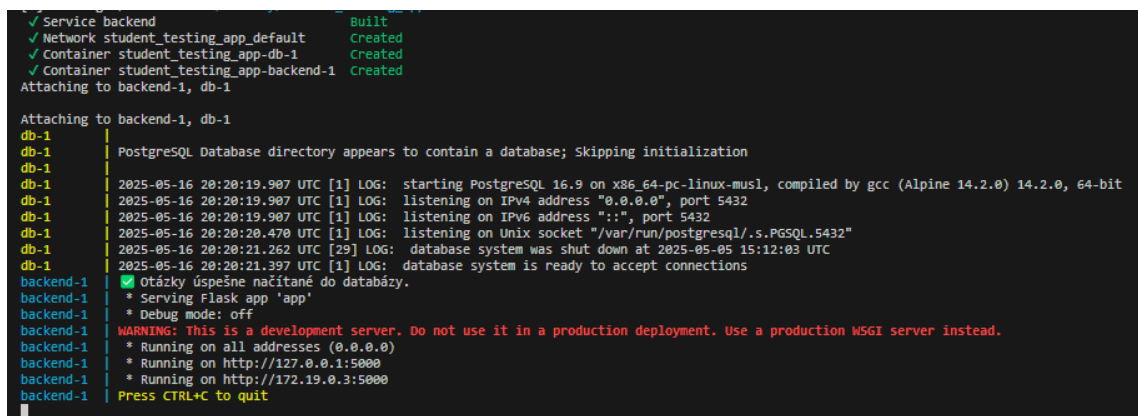
1. Otvorte terminál (príkazový riadok) v koreňovom adresári projektu, kde sa nachádzajú súbory `docker-compose.yml` a `Dockerfile`.
2. Spustíte nasledujúci príkaz na vytvorenie a spustenie kontajnerov:

```
$ docker-compose up --build
```

3. Po chvíli sa zobrazí obrazovka terminálu ako na obr. 6–1, čo znamená, že:

- kontajner databázy `db-1` používa port **5432** pre prístup k databáze,
- server aplikácie `backend-1` pracuje a je prístupný na:

`http://127.0.0.1:5000`



```
✓ Service backend Built
✓ Network student_testing_app_default Created
✓ Container student_testing_app-db-1 Created
✓ Container student_testing_app-backend-1 Created
Attaching to backend-1, db-1

Attaching to backend-1, db-1
db-1 | PostgreSQL Database directory appears to contain a database; Skipping initialization
db-1 |
db-1 | 2025-05-16 20:20:19.907 UTC [1] LOG: starting PostgreSQL 16.9 on x86_64-pc-linux-musl, compiled by gcc (Alpine 14.2.0) 14.2.0, 64-bit
db-1 | 2025-05-16 20:20:19.907 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
db-1 | 2025-05-16 20:20:19.907 UTC [1] LOG: listening on IPv6 address ":::", port 5432
db-1 | 2025-05-16 20:20:20.470 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2025-05-16 20:20:21.262 UTC [29] LOG: database system was shut down at 2025-05-05 15:12:03 UTC
db-1 | 2025-05-16 20:20:21.397 UTC [1] LOG: database system is ready to accept connections
backend-1 | ✓ Otázky úspešne načítané do databázy.
backend-1 | * Serving Flask app 'app'
backend-1 | * Debug mode: off
backend-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
backend-1 | * Running on all addresses (0.0.0.0)
backend-1 | * Running on http://127.0.0.1:5000
backend-1 | * Running on http://172.19.0.3:5000
backend-1 | Press CTRL+C to quit
```

Obr. 6–1 Výpis terminálu po spustení kontajnerov

Týmto spôsobom spustíte aplikáciu bez nutnosti manipulovať so závislosťami a knižnicami, pričom je celé prostredie, vďaka technológii Docker, izolované.

Na vývojové alebo ladiace účely sa môžete dokonca dostať k jednotlivým komponentom nasledovne:

Zobrazenie bežiacich kontajnerov:

```
$ docker ps
```

Pripojenie sa k už bežiacemu backend kontajneru:

```
$ docker exec -it <nazov_backend_kontajnera> /bin/bash
```

Ako sa pripojiť k databáze priamo z kontajnera:

```
$ docker exec -it <nazov_db_kontajnera> psql -U postgres -d  
adaptive_db
```

Zoznam použitej literatúry

- [1] MITTAL, B. 2023. *Python Programming Questions Dataset*. Dostupné na internete: <https://www.kaggle.com/datasets/bhavesmittal/python-programming-questions-dataset> [cit. 2025-04-26].
- [2] DOCKER. 2025. *Docker Documentation*. Dostupné na internete: <https://docs.docker.com/> [cit. 2025-05-15].
- [3] POSTGRESQL. 2025. *PostgreSQL Documentation*. Dostupné na internete: <https://www.postgresql.org/docs/> [cit. 2025-05-15].
- [4] FLASK. 2018. *Flask Documentation*. Dostupné na internete: <https://flask.palletsprojects.com/en/stable/> [cit. 2025-05-15].